

FALCON PLAYER

Complete Project Architecture, Engineering Reference & Project-Based Learning Textbook

Target Codebase: com.example.falconplayer

Platform & Engine: Android SDK 36 • AndroidX Media3 (ExoPlayer 1.7.0) • Jetpack Compose Material 3

Architecture Pattern: Unidirectional Data Flow (UDF) • MVVM • Dagger Hilt DI • AndroidX Navigation 3

Author / Role: Senior Android Systems Architect & Technical Writer

Document Classification: Complete Technical Specification & Step-by-Step Developer Manual

A comprehensive, source-grounded guide to developing, debugging, maintaining, and extending the Falcon Player Android codebase.

Compiled from direct static inspection of the Falcon Player production repository.

Table of Contents

Chapter	Synopsis & Key Topics
Chapter 1 - Project Overview	High-level architecture, feature matrix, and system flow
Chapter 2 - Complete Project Structure	Directory tree and prioritized analysis of top 20 core source files
Chapter 3 - Programming Foundations Used in Falcon Player	Data classes, sealed interfaces, extension functions, coroutines
Chapter 4 - Android Fundamentals Through Falcon Player	Activities, window insets, configChanges, and scoped storage
Chapter 5 - Application Startup and Navigation	AndroidX Navigation 3, NavKey routes, and startup lifecycle trace
Chapter 6 - Complete UI and Design System	Color tokens, typography, Falcon logo Canvas drawing, and layouts
Chapter 7 - Media Discovery and Library Management	MediaStore queries, resolution badges, and scoped storage deletes
Chapter 8 - Video Playback Engine: Deep Technical Explanation	Media3 ExoPlayer, PlayerView surface, and accumulating seek
Chapter 9 - Audio Playback and Audio Library	AudioPlaybackManager, queue management, audio focus, noise safety
Chapter 10 - Audio Tracks and Subtitle System	Track discovery, technical codec badges, and selection overrides
Chapter 11 - Timeline, Controls, Gestures, and Fullscreen	Custom scrubbing Canvas, brightness/volume HUDs, orientation fix
Chapter 12 - Android Permissions, Security, and Storage	Granular API 33+ permissions, Incognito Mode, and storage boundaries
Chapter 13 - Architecture and Data Flow	Unidirectional Data Flow (UDF), StateFlow, and Mutex persistence
Chapter 14 - Dependencies and External Libraries	libs.versions.toml version catalog and dependency impact analysis
Chapter 15 - Build System and APK Distribution	Gradle build scripts, compileSdk 36, and ADB deployment commands
Chapter 16 - Debugging and Troubleshooting	Logcat tags, playback error diagnosis, and leak prevention
Chapter 17 - Source-Code Reference	Detailed method signatures, side effects, and caller dependencies
Chapter 18 - Project-Based Learning Course	6 progressive developer stages (A to F) with practical exercises
Chapter 19 - Feature Status and Technical Debt	Source-confirmed feature matrix and architectural debt analysis
Chapter 20 - Glossary, FAQ, and Final Reference	50 project-specific questions & answers and developer mastery roadmap

Chapter 1 - Project Overview

1.1 Executive Summary & Mission

Falcon Player (package `com.example.falconplayer`) is a state-of-the-art, high-performance multimedia playback application for Android. Engineered entirely using **Jetpack Compose** (Material 3), Google's cutting-edge **AndroidX Media3 (ExoPlayer 1.7.0)** engine, and **Dagger Hilt** dependency injection, Falcon Player provides an uncompromising local playback experience for high-definition video and lossless audio.

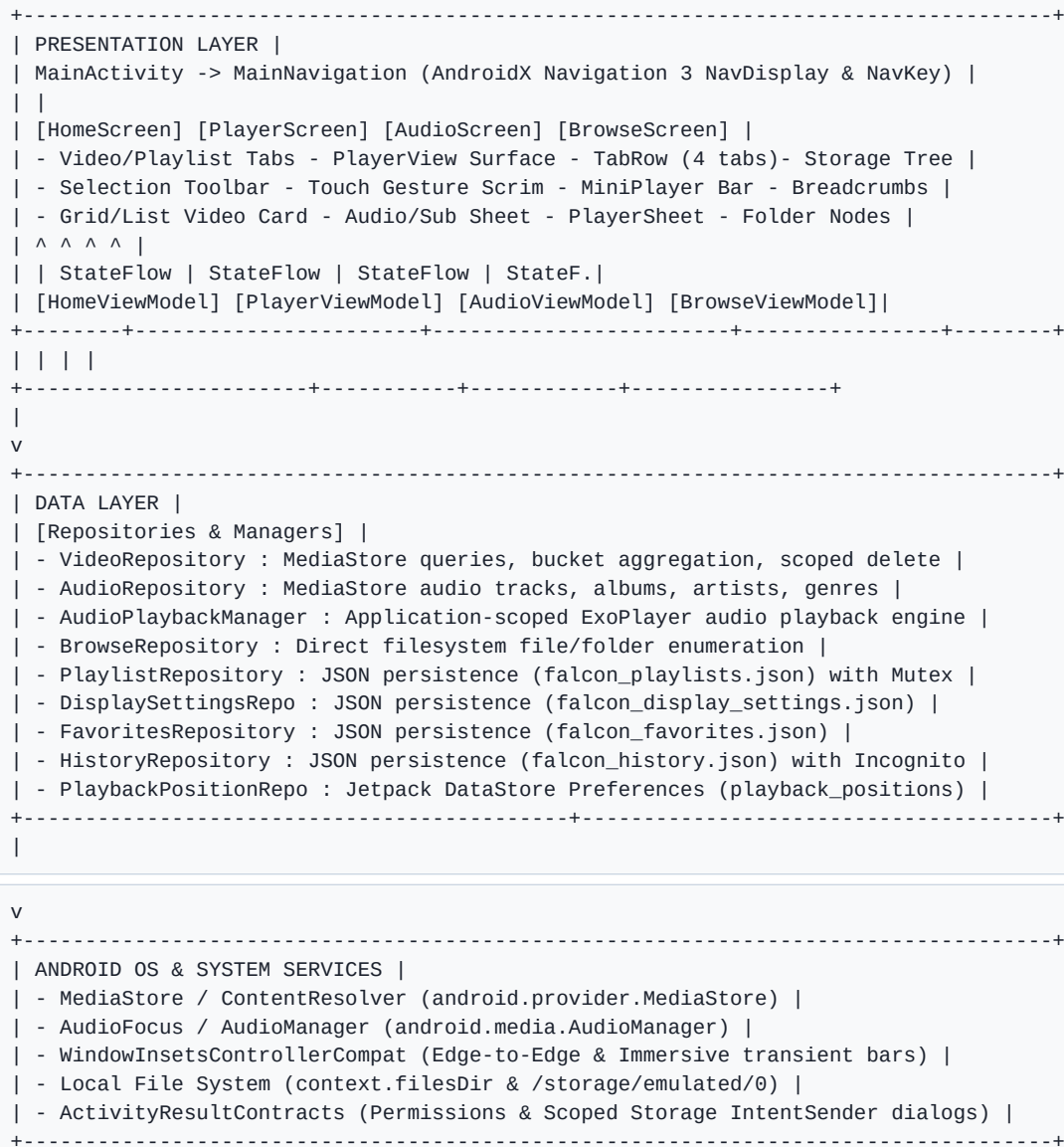
Unlike generic Android media players that wrap legacy Android Views or rely on outdated ExoPlayer v2 APIs, Falcon Player is built from the ground up on modern declarative Android development paradigms. It features hardware-accelerated video rendering with custom interactive gestures, immersive system bar management, scoped storage discovery and deletion, multi-track audio and subtitle selection, background music queue management, and a custom aerodynamic branding system.

1.2 Core Capabilities & Feature Map

Functional Area	Core Capabilities Implemented	Underlying Technology
Video Playback Engine	Hardware-accelerated decoding, aspect ratio switching (Fit/Zoom/Stretch), dynamic playback speed (0.25x - 2.0x), double-tap seek with time accumulation, vertical drag brightness & volume HUDs, picture-in-picture mode, timeline scrubbing with buffer display.	AndroidX Media3 ExoPlayer 1.7.0, PlayerView, Compose AndroidView interop
Audio Player & Library	Categorized music browsing (Artists, Albums, Tracks, Genres), persistent playback queue, shuffle & repeat modes (Off, All, One), floating expandable Mini Player, full-screen playback sheet with album art, automatic audio focus management, and headphone disconnection safety.	Media3 ExoPlayer, AudioManager, ContentResolver thumbnail loader
Media Discovery	Zero-lag MediaStore video & audio scanning, automatic bucket/folder grouping, resolution badge computation (4K, 1080p, 720p, 480p), memory-efficient LruCache thumbnail extraction with ContentResolver and MediaMetadataRetriever fallbacks.	MediaStore.Video/Audio, LruCache, Kotlin Coroutines Dispatchers.IO
Scoped Storage & Deletion	Full compatibility with Android 11+ (API 30+) scoped storage delete requests, Android 10 (API 29) RecoverableSecurityException handling, single-item deletion, long-press multi-selection, and atomic bulk file deletion.	MediaStore.createDeleteRequest, IntentSender, ActivityResultContracts
Playlists & Organization	Custom video playlists with validation, unique ID generation, add/remove videos, drag-and-drop reordering, JSON file persistence with thread-safe Mutex locking, search filtering, and incognito mode history exclusion.	Kotlinx Serialization, internal storage JSON, Kotlin Mutex
File Explorer (Browse)	Direct file system exploration, primary internal storage traversal, secondary SD card detection, quick-access shortcuts (Download, Movies, Music), and media file counting.	Java File API, Environment.getExternalStorageDirectory

1.3 High-Level System Architecture & Layering

Falcon Player follows a decoupled **Unidirectional Data Flow (UDF)** layered architecture with Clean Architecture principles adapted for modern Jetpack Compose applications:



1.4 What Happens When the Application Launches

When the user taps the Falcon Player icon on their device home screen, the Android system triggers a precise initialization cascade:

Step 1: Application Subclass Initialization. The operating system instantiates `FalconPlayerApp`, which is annotated with `@HiltAndroidApp`. Dagger Hilt builds the dependency graph and registers singleton providers in memory.

Step 2: Activity Creation & Edge-to-Edge Configuration. The system launches `MainActivity` (annotated with `@AndroidEntryPoint`). Inside `onCreate()`, `enableEdgeToEdge()` is invoked, and `WindowCompat.getInsetsController()`

sets `BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE`, hiding system status and navigation bars for an edge-to-edge cinematic canvas.

Step 3: Compose Root Attachment. `setContent` renders `FalconPlayerTheme` wrapping `MainNavigation()`. Navigation 3 initializes `rememberNavController()` with the Home `NavKey`.

Step 4: Permission Verification & Media Scanning. `HomeScreen` queries storage permissions. On Android 13+ (API 33+), it requests `READ_MEDIA_VIDEO` and `READ_MEDIA_AUDIO`; on older platforms, it requests `READ_EXTERNAL_STORAGE`. Once granted, `HomeViewModel.loadMedia()` executes a background `MediaStore` query via `Dispatchers.IO`, extracts video metadata, builds folder groupings, and emits the data via `HomeUiState` to render the media grid.

MODERN ARCHITECTURE HIGHLIGHT: Falcon Player completely avoids deprecated Android APIs. It uses AndroidX Navigation 3 (`NavController` and `NavKey`) instead of legacy string-based route parsing, AndroidX Media3 instead of legacy `com.google.android.exoplayer2`, and Jetpack DataStore Preferences instead of legacy `SharedPreferences`.

Chapter 2 - Complete Project Structure

2.1 Codebase File Tree

The complete source structure of the Falcon Player application demonstrates clean separation between data modeling, storage repositories, UI presentation, design tokens, and components:

```
c:\falcon_player\
■■■ app/
■ ■■■ build.gradle.kts # Module build script, dependencies, SDK configuration
■ ■■■ src/
■ ■ ■■■ main/
■ ■ ■ ■■■ AndroidManifest.xml # Manifest permissions, activity & hardware configs
■ ■ ■ ■■■ java/com/example/falconplayer/
■ ■ ■ ■ ■■■ FalconPlayerApp.kt # Application class with @HiltAndroidApp
■ ■ ■ ■ ■■■ MainActivity.kt # ComponentActivity entry point, immersive controller
■ ■ ■ ■ ■■■ Navigation.kt # Navigation 3 NavDisplay & entryProvider
■ ■ ■ ■ ■■■ NavigationKeys.kt # NavKey serializable routes (Home, Player, PlaylistDetail)
■ ■ ■ ■ ■■■ data/
■ ■ ■ ■ ■■■ ■■■ MediaItem.kt # VideoItem, FolderItem, AudioItem, ArtistItem, AlbumItem
■ ■ ■ ■ ■■■ ■■■ Playlist.kt # Playlist data model (id, name, videoUris)
■ ■ ■ ■ ■■■ ■■■ BrowseItem.kt # BrowseFolderItem, BrowseFileItem, BrowseNodeItem
■ ■ ■ ■ ■■■ ■■■ VideoRepository.kt # MediaStore video queries, folders, scoped delete
■ ■ ■ ■ ■■■ ■■■ AudioRepository.kt # MediaStore audio tracks, artists, albums, genres
■ ■ ■ ■ ■■■ ■■■ AudioPlaybackManager.kt # Singleton ExoPlayer for audio, queue, audio focus
■ ■ ■ ■ ■■■ ■■■ BrowseRepository.kt # File system directory scanner & storage root finder
■ ■ ■ ■ ■■■ ■■■ PlaylistRepository.kt # JSON file storage for playlists, CRUD, validation
■ ■ ■ ■ ■■■ ■■■ DisplaySettingsRepository.kt # JSON storage for view modes & sort orders
■ ■ ■ ■ ■■■ ■■■ FavoritesRepository.kt # JSON storage for favorite media URIs
■ ■ ■ ■ ■■■ ■■■ HistoryRepository.kt # JSON storage for playback history & incognito mode
■ ■ ■ ■ ■■■ ■■■ PlaybackPositionRepository.kt # DataStore Preferences for resume positions
■ ■ ■ ■ ■■■ ■■■ theme/
■ ■ ■ ■ ■■■ ■■■ Color.kt # FalconRed (#E50914), Dark surfaces, text tokens
■ ■ ■ ■ ■■■ ■■■ Theme.kt # FalconPlayerTheme with darkColorScheme
■ ■ ■ ■ ■■■ ■■■ Type.kt # Typography system
■ ■ ■ ■ ■■■ ■■■ ui/
■ ■ ■ ■ ■■■ ■■■ audio/
```

```
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ AudioScreen.kt # 4-tab audio library (Artists, Albums, Tracks, Genres)
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ AudioViewModel.kt # Audio state management & search filtering
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ AudioMiniPlayer.kt # Persistent floating mini player bar
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ AudioPlayerSheet.kt # Expandable full-screen music player sheet
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ browse/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ BrowseScreen.kt # File manager interface with storage disks
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ BrowseViewModel.kt# Directory navigation stack & search
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ FalconLogo.kt # Custom vector Canvas drawing of brand falcon
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ThumbnailLoader.kt# Memory LruCache video thumbnail bitmap extractor
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ home/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ HomeScreen.kt # Main container with 5-tab bottom bar & video grid
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ HomeViewModel.kt # Media scanning, sorting, multi-select bulk delete
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ DisplaySettingsScreen.kt # Sort & display customization screen
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ MoreScreen.kt # Settings, history, and app about dialog
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ player/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerScreen.kt # Media3 PlayerView, gestures, overlays, HUDs
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerViewModel.kt# ExoPlayer controller, seek, tracks, speed, queue
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerUiState.kt # State models, TrackItem, VideoResizeMode
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlaybackStatusIndicator.kt # Animated status pill
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerBottomControls.kt # Bottom action bar & speed selector
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerCenterControls.kt # Centered minimal play/skip cluster
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerControls.kt # Animated visibility parent overlay
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerSettingsSheet.kt # Playback speed & info bottom sheet
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlayerTopBar.kt # Back button, title, track buttons
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ TrackSelectionSheet.kt # Audio & subtitle selector sheet
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ VideoInfoDialog.kt # Technical video codec metadata dialog
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ VideoTimeline.kt # Custom Canvas interactive scrubbing bar
```

```
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ playlist/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlaylistDetailScreen.kt # Playlist tracks, duration & reordering
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlaylistViewModel.kt # Playlist state & dialog triggers
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ PlaylistDialogs.kt # Create, Rename, Delete, AddTo dialogs
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ res/ # XML resources, drawables, launcher icons, themes
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ test/ # Unit tests (PlayerUiStateTest.kt)
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ gradle/
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ libs.versions.toml # Version catalog with pinned dependency versions
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ build.gradle.kts # Root project build file
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ settings.gradle.kts # Repository management and module inclusion
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ gradle.properties # JVM arguments and AndroidX flags
```

2.2 Prioritized Analysis of the Top 20 Architectural Files

To master, maintain, and extend Falcon Player, developers should study these 20 foundational files in the recommended sequence:

Priority & File Path	Responsibility & Key Members	Impacted Features & Next Study Target
#1 data/VideoRepository.kt	Queries MediaStore for video rows; aggregates by bucket/folder; executes scoped storage deletes via createDeleteRequest.	Affects video library display, folder browsing, file deletion. Next: HomeViewModel.kt
#2 ui/player/PlayerViewModel.kt	Manages ExoPlayer instance; track selection (audio/subs); double-tap seeking; speed regulation; position saving.	Core video playback engine, controls visibility, resume playback. Next: PlayerScreen.kt

#3 ui/player/PlayerScreen.kt	Renders Media3 PlayerView; vertical drag gestures for brightness/volume; double-tap seek detection; bottom sheets.	Player UI, gesture response, PiP, orientation changes. Next: PlayerControls.kt
#4 data/AudioPlaybackManager.kt	Singleton ExoPlayer engine for background audio; queue management; audio focus handling; headphone noise disconnection.	All audio playback, queue shuffle/repeat, mini player. Next: AudioViewModel.kt
#5 ui/home/HomeViewModel.kt	Mediates between VideoRepository and HomeScreen; implements search, sorting, folder filtering, and multi-select bulk delete.	Video library UI, sorting, search, bulk deletion. Next: HomeScreen.kt
#6 ui/home/HomeScreen.kt	Main screen scaffold; 5-tab bottom navigation bar; floating action button; selection mode toolbar; list/grid toggle.	App home navigation, tab switching, video card list. Next: Navigation.kt
#7 Navigation.kt	Configures AndroidX Navigation 3; defines NavDisplay and entryProvider mapping NavKeys to Compose screens.	Entire application navigation hierarchy and backstack. Next: NavigationKeys.kt
#8 NavigationKeys.kt	Defines type-safe, serializable navigation routes: Home, PlaylistDetail, and Player.	Screen routing parameters and deep links. Next: MainActivity.kt
#9 data/AudioRepository.kt	Queries MediaStore for audio files; parses artists, albums, and genres; resolves album artwork content URIs.	Music library data, artist/album categorization. Next: AudioScreen.kt
#10 ui/player/components/VideoTimeline.kt	Custom Canvas timeline scrubber; renders elapsed, buffered, and remaining time; handles scrubbing drag gestures.	Playback seeking precision and visual progress. Next: PlayerBottomControls.kt
#11 ui/player/components/TrackSelectionSheet.kt	Modal bottom sheet for selecting audio languages and subtitle tracks; provides instant codec details and disable option.	Multi-language audio & subtitle switching. Next: PlayerSettingsSheet.kt
#12 data/PlaylistRepository.kt	Thread-safe JSON persistence for custom user playlists; validates playlist names; handles reordering and deletions.	Playlist persistence, playlist creation & editing. Next: PlaylistViewModel.kt
#13 data/BrowseRepository.kt	File system scanner; discovers internal storage and removable SD cards; filters playable media files.	File browser tab, folder navigation, direct file playback. Next: BrowseScreen.kt
#14 ui/components/ThumbnailLoader.kt	Global in-memory LruCache for video thumbnail bitmaps; leverages ContentResolver.loadThumbnail with fallbacks.	Media card thumbnail rendering performance and memory safety. Next: RealVideoGridCard
#15 data/PlaybackPositionRepository.kt	Manages Jetpack DataStore Preferences for saving and restoring exact video playback millisecond timestamps.	Resume playback functionality across sessions. Next: PlayerViewModel.kt
#16 data/MediaItem.kt	Fundamental domain entities: VideoItem, AudioItem, FolderItem, ArtistItem, AlbumItem.	All data layers and UI representations. Next: VideoRepository.kt
#17 ui/audio/components/AudioPlayerSheet.kt	Full-screen expandable audio sheet with artwork, queue list, shuffle/repeat toggles, and playback speed pill.	Now playing music experience, queue inspection. Next: AudioMiniPlayer.kt
#18 ui/home/DisplaySettingsScreen.kt	Customization screen allowing users to toggle list/grid layouts, filter favorites, and switch sort orders.	User preferences, sorting algorithms, favorites filter. Next: DisplaySettingsRepository.kt

#19 MainActivity.kt	Activity entry point; enables edge-to-edge layout; configures transient system bars; mounts root Compose tree.	App startup, window insets, system bar behavior. Next: AndroidManifest.xml
#20 ui/components/FalconLogo.kt	Pure Compose Canvas implementation of the Falcon brand logo; computes geometric feathers, aerodynamic beak, and eye highlight.	Branding aesthetics, vector drawing techniques. Next: Theme.kt

Chapter 3 - Programming Foundations Used in Falcon Player

3.1 Core Kotlin Concepts Encountered in the Codebase

To develop and modify Falcon Player confidently, you must understand the exact Kotlin programming language features utilized across its components. Rather than abstract theory, each concept below is explained directly through actual project code.

Concept 1: Data Classes and Immutability

A data class in Kotlin is a specialized class designed to hold immutable state. The Kotlin compiler automatically generates `equals()`, `hashCode()`, `toString()`, and the crucial `copy()` method.

```
// Location: app/src/main/java/com/example/falconplayer/data/MediaItem.kt
data class VideoItem(
    val id: Long,
    val contentUri: Uri,
    val title: String,
    val durationMs: Long,
    val width: Int,
    val height: Int,
    val sizeBytes: Long,
    val bucketId: String,
    val bucketName: String,
    val dateAddedSec: Long = 0L,
    val resolutionBadge: String? = null,
    val durationFormatted: String = formatDuration(durationMs)
)
```

Line-by-Line Explanation:

- `val` declares read-only properties, preventing accidental mutation of video records in memory.
- Default arguments like `dateAddedSec = 0L` allow callers to instantiate the class without providing every field.
- `durationFormatted` uses a computed default value: whenever a `VideoItem` is created, `formatDuration(durationMs)` is executed immediately to produce a formatted timestamp like "04:12".
- **Where it appears:** `MediaItem.kt`, `Playlist.kt`, `PlayerUiState.kt`, `BrowseItem.kt`.
- **Student Exercise:** Add a new property `val fileExtension: String` to `VideoItem` that extracts the lowercase extension from `title`.

Concept 2: Sealed Interfaces & Exhaustive Pattern Matching

A sealed interface restricts hierarchy inheritance to the file in which it is declared. This allows the Kotlin compiler to enforce **exhaustive when expressions** without requiring an `else` branch.

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerUiState.kt
sealed interface PlaybackState {
    object Idle : PlaybackState
    object Buffering : PlaybackState
    object Playing : PlaybackState
    object Paused : PlaybackState
    object Ended : PlaybackState
    data class Error(val message: String) : PlaybackState
}

val PlaybackState.statusText: String
get() = when (this) {
    PlaybackState.Buffering -> "Loading"
    PlaybackState.Playing -> "Playing"
    PlaybackState.Paused, PlaybackState.Idle, PlaybackState.Ended -> "Paused"
    is PlaybackState.Error -> "Error"
}
```

Line-by-Line Explanation:

- PlaybackState represents all possible operational states of the ExoPlayer engine.
- object Idle is a memory-efficient singleton because it carries no dynamic state.
- data class Error(val message: String) can encapsulate variable payloads such as a decoder exception.
- In the extension property statusText, the when expression checks every subtype. If a developer adds a new state (e.g. object Stalled) in the future, the compiler will refuse to compile until that state is handled.
- **Where it appears:** PlayerUiState.kt (PlaybackState, ControlsState), VideoRepository.kt (DeleteResult), BrowseItem.kt (BrowseNodeItem).
- **Student Exercise:** Write a when statement that takes a PlaybackState and returns a boolean indicating whether the center play button should show a Pause icon.

Concept 3: Coroutines, Dispatchers, and StateFlow

Falcon Player executes all disk and database I/O asynchronously off the main thread using Kotlin Coroutines, and broadcasts observable UI state using StateFlow.

```
// Location: app/src/main/java/com/example/falconplayer/data/VideoRepository.kt
suspend fun getVideos(): List<VideoItem> = withContext(Dispatchers.IO) {
    val videos = mutableListOf<VideoItem>()
    context.contentResolver.query(
        MediaStore.Video.Media.EXTERNAL_CONTENT_URI,
        projection,
        null, null, null
    )?.use { cursor ->
        while (cursor.moveToNext()) {
            // Read columns and add to list...
        }
    }
    videos
}
```

Line-by-Line Explanation:

- suspend marks a function that can pause execution without blocking the calling thread.
- withContext(Dispatchers.IO) switches execution to an optimized thread pool dedicated to disk and database operations, keeping the UI perfectly responsive at 120 FPS.
- cursor.use { } is an extension function that automatically closes the Cursor when the block finishes or throws an exception, preventing memory leaks.

- **Student Exercise:** Trace how `HomeViewModel.loadMedia()` calls `videoRepository.getVideos()` inside `viewModelScope.launch` and updates `_uiState.update { it.copy(...) }`.

Chapter 4 - Android Fundamentals Through Falcon Player

4.1 Manifest Declarations & Hardware Configuration

The AndroidManifest.xml defines the contract between Falcon Player and the Android operating system. Inspecting the actual manifest reveals key architectural decisions:

```
<!-- Location: app/src/main/AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
  <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" android:maxSdkVersion="32" />
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" android:maxSdkVersion="29" />
  <uses-permission android:name="android.permission.MANAGE_MEDIA" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

  <application
    android:name=".FalconPlayerApp"
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.FalconPlayer">
    <activity
      android:name=".MainActivity"
      android:exported="true"
      android:supportsPictureInPicture="true"
      android:configChanges="screenSize|smallestScreenSize|screenLayout|orientation">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

4.2 Why configChanges is Critical for Video Playback

Notice the attribute: `android:configChanges="screenSize|smallestScreenSize|screenLayout|orientation"`. In standard Android applications, rotating the device from portrait to landscape destroys and recreates the entire Activity. In a high-end video player, recreating the Activity would cause the video decoder to reset, stutter, re-buffer, and lose surface rendering.

By declaring these `configChanges`, Falcon Player instructs Android to **retain the existing Activity instance** during orientation and layout shifts. The Jetpack Compose UI seamlessly recalculates its layout constraints, while the underlying ExoPlayer engine continues decoding without dropping a single frame.

4.3 Immersive Fullscreen and Window Insets

In MainActivity.kt, Falcon Player suppresses the Android system bars (status bar and navigation pill) to give videos an edge-to-edge canvas:

```
// Location: app/src/main/java/com/example/falconplayer/MainActivity.kt
enableEdgeToEdge()

val windowInsetsController = WindowCompat.getInsetsController(window, window.decorView)
windowInsetsController.systemBarsBehavior =
    WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
windowInsetsController.hide(WindowInsetsCompat.Type.systemBars())
```

Behavioral Impact: BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE allows the user to swipe from the screen edge to reveal transient system bars that automatically fade out after a brief interval, ensuring that playback controls remain undisturbed.

Chapter 5 - Application Startup and Navigation

5.1 AndroidX Navigation 3 Architecture

Falcon Player utilizes the latest **AndroidX Navigation 3** framework (androidx.navigation3:navigation3-runtime:1.0.1 and navigation3-ui). Unlike legacy Navigation Compose which relied on fragile string path concatenation (such as "player/{videoid}?title={title}"), Navigation 3 is built upon type-safe, compile-time verified **NavKey** objects.

```
// Location: app/src/main/java/com/example/falconplayer/NavigationKeys.kt
package com.example.falconplayer

import androidx.navigation3.runtime.NavKey
import kotlin.serialization.Serializable

@Serializable
data object Home : NavKey

@Serializable
data class PlaylistDetail(val playlistId: String) : NavKey

@Serializable
data class Player(
    val videoUri: String? = null,
    val title: String? = null,
    val playlistId: String? = null,
    val playlistIndex: Int = 0
) : NavKey
```

5.2 Navigation Controller & NavDisplay Implementation

In Navigation.kt, the MainNavigation Composable initializes the navigation stack and registers the screen entry providers:

```
// Location: app/src/main/java/com/example/falconplayer/Navigation.kt
@Composable
fun MainNavigation() {
    val backStack = rememberNavBackStack(Home)

    NavDisplay(
        backStack = backStack,
        onBack = { backStack.removeLastOrNull() },
        entryProvider = entryProvider {
            entry<Home> {
                HomeScreen(
                    onPlayMedia = { uri, title ->
                        backStack.add(Player(videoUri = uri.toString(), title = title))
                    },
                    onOpenPlaylist = { playlistId ->
                        backStack.add(PlaylistDetail(playlistId = playlistId))
                    }
                )
            }
            entry<PlaylistDetail> { key ->
                PlaylistDetailScreen(
                    playlistId = key.playlistId,
                    onBackClick = { backStack.removeLastOrNull() },
                    onPlayVideoInPlaylist = { uri, title, playlistId, index ->
                        backStack.add(Player(uri.toString(), title, playlistId, index))
                    }
                )
            }
            entry<Player> { key ->
                PlayerScreen(

```

```
                    onBackClick = { backStack.removeLastOrNull() },
                    initialVideoUri = key.videoUri?.let { Uri.parse(it) },
                    initialTitle = key.title,
                    initialPlaylistId = key.playlistId,
                    initialPlaylistIndex = key.playlistIndex
                )
            }
        }
    )
}
```

5.3 Complete Sequence Diagram: Video Selection to Playback

```
User HomeScreen backStack NavDisplay PlayerScreen PlayerViewModel
| | | | | | | |
|-- Tap Video Card ---->| | | |
| |-- backStack.add(Player)-> | | |
| | |-- Push Key ----->| | |
| | | |-- Mount Screen -->| | |
| | | | |-- loadVideo(uri) -->| | |
| | | | | |-- MediaItem.fromUri
| | | | | |-- player.prepare()
| | | | | |-- seekTo(savedPos)
| | | | | |-- player.play()
| | | | |<-- Buffering -----|
| | | |<-- Playing -----| |
|<-- Render Video Frame -----+ |
```

Chapter 6 - Complete UI and Design System

6.1 Color Palette & Visual Tokens

Falcon Player enforces a high-contrast cinematic dark theme. The color system is centralized in theme/Color.kt and bound to Material 3 in theme/Theme.kt:

Token Name	Hex Value	Role & Usage Across UI
FalconRed	#E50914	Primary brand accent; active playback indicators, scrubbers, FABs, selected tabs.
FalconRedDark	#B81D24	Container highlights, gradient end-stops in logo feathers.
FalconRedLight	#FF3B30	Secondary badges, gradient start-stops for vivid highlights.
FalconBackground	#101010	Deep near-black application scaffold background, minimizing battery drain on OLED screens.
FalconSurface	#1E1E1E	Elevated cards, bottom sheets, navigation bar containers, dialog backdrops.
FalconSurfaceVariant	#282828	Indicator pills, secondary buttons, subtle dividers.
FalconTextPrimary	#FFFFFF	High-emphasis video titles, active tab labels, timestamp digits.
FalconTextSecondary	#A0A0A0	Medium-emphasis file sizes, folder counts, artist names, unselected tabs.

6.2 Vector Branding: The Falcon Logo Canvas

Rather than loading a static raster image, Falcon Player's logo in ui/components/FalconLogo.kt is procedurally rendered using Compose Canvas. It constructs 6 mathematically proportioned vector paths:

- Upper Wing Feather:** Path in dark crimson (#B3121C).
- Middle Wing Feather:** Path in mid crimson (#D81722).
- Lower Wing Feather:** Path in deep shadow red (#960E16).
- Aerodynamic Beak (Play Triangle):** Dynamic forward play arrow rendered with a linear gradient spanning #FF3843 to #B80710.
- Crest Highlight:** Translucent white overlay (Color.White.copy(alpha = 0.22f)) simulating specular reflection.
- Piercing White Eye:** Centered polygon highlighting the playhead core.

Chapter 7 - Media Discovery and Library Management

7.1 Local Video Discovery via MediaStore

Falcon Player queries the Android MediaStore.Video.Media.EXTERNAL_CONTENT_URI database through a robust projection defined in VideoRepository.kt:

```
// Location: app/src/main/java/com/example/falconplayer/data/VideoRepository.kt
val projection = arrayOf(
    MediaStore.Video.Media._ID,
    MediaStore.Video.Media.DISPLAY_NAME,
    MediaStore.Video.Media.TITLE,
    MediaStore.Video.Media.DURATION,
    MediaStore.Video.Media.WIDTH,
    MediaStore.Video.Media.HEIGHT,
    MediaStore.Video.Media.SIZE,
    MediaStore.Video.Media.BUCKET_ID,
    MediaStore.Video.Media.BUCKET_DISPLAY_NAME,
    MediaStore.Video.Media.DATA,
    MediaStore.Video.Media.DATE_ADDED
)
```

Handling Missing or Null Metadata: Android devices vary widely in how MediaStore indexes media. Falcon Player implements defensive fallbacks:

- **Title Fallback:** `val name = displayName ?: title ?: dataPath?.let { File(it).name } ?: "Video_$id"`
- **Folder Bucket Fallback:** If `BUCKET_DISPLAY_NAME` is null, the repository parses `File(dataPath).parentFile.name`. If `BUCKET_ID` is null, it hashes the bucket name: `bucketName.hashCode().toString()`.
- **Resolution Badge Calculation:** Dynamically inspects width and height to append 4K ($\geq 2160p$), 1080p, 720p, or 480p badges.

7.2 Scoped Storage Deletion & Android OS Dialogs

Android Scoped Storage rules (Android 10+) strictly prohibit an application from directly calling `File.delete()` on media files owned by other apps. Falcon Player implements a three-tier deletion architecture in `VideoRepository.deleteVideo()`:

```
// Location: app/src/main/java/com/example/falconplayer/data/VideoRepository.kt
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
    // Android 11+ (API 30+) - Atomically requests OS permission dialog
    val intentSender = MediaStore.createDeleteRequest(
        context.contentResolver,
        listOf(videoUri)
    ).intentSender
    DeleteResult.NeedsPermission(intentSender)
} else if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
    // Android 10 (API 29) - Catch RecoverableSecurityException
    try {
        val deleted = context.contentResolver.delete(videoUri, null, null)
        if (deleted > 0) DeleteResult.Success else DeleteResult.Failure("Not found")
    } catch (e: android.app.RecoverableSecurityException) {
        DeleteResult.NeedsPermission(e.userAction.actionIntent.intentSender)
    }
} else {
    // Android 9 and below - Direct delete
    val deleted = context.contentResolver.delete(videoUri, null, null)
    if (deleted > 0) DeleteResult.Success else DeleteResult.Failure("Not found")
}
```

7.3 Multi-Selection and Bulk Deletion

In HomeScreen.kt, users can long-press any video card to enter **Selection Mode** (isSelectionMode = true). Multiple video IDs are collected in a Set<Long>.

When the user taps **Delete Selected**, HomeViewModel.deleteSelectedVideos() passes all selected URIs to VideoRepository.deleteVideos(uris). On Android 11+, MediaStore.createDeleteRequest(contentResolver, uris) displays a **single system dialog** confirming the deletion of all selected items simultaneously.

Chapter 8 - Video Playback Engine: Deep Technical Explanation

8.1 Player Initialization & Surface Interoperability

Falcon Player instantiates its video engine via `ExoPlayer.Builder(context).build()` inside `PlayerViewModel.kt`. To embed the video surface inside a declarative Compose hierarchy, `PlayerScreen.kt` uses `AndroidView` wrapping Media3's `PlayerView`:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerScreen.kt
AndroidView(
    factory = { ctx ->
        PlayerView(ctx).apply {
            player = viewModel.player
            useController = false // Custom Compose controls used instead
            resizeMode = uiState.resizeMode.mode
            subtitleView?.setUserDefaultStyle()
            subtitleView?.setFractionalTextSize(0.053f)
        }
    },
    update = { playerView ->
        playerView.player = viewModel.player
        if (playerView.resizeMode != uiState.resizeMode.mode) {
            playerView.resizeMode = uiState.resizeMode.mode
        }
    },
    modifier = Modifier.fillMaxSize()
)
```

Setting `useController = false` instructs Media3 to hide its default XML playback controls, allowing Falcon Player's custom Compose overlays to manage the entire user interface.

8.2 Double-Tap Seeking with Cumulative Time Aggregation

A critical feature in Falcon Player is **accumulating double-tap seek feedback**. Rapid successive taps continue adding +10s (+10s, +20s, +30s) rather than resetting or flashing:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerViewModel.kt
fun triggerDoubleTapSeek(isForward: Boolean) {
    if (doubleTapLastIsForward != isForward) {
        doubleTapSeekAccumulatedMs = 0L // Reset if direction changed
    }
    doubleTapLastIsForward = isForward
    doubleTapSeekAccumulatedMs += 10_000L

    val seekDeltaMs = if (isForward) 10_000L else -10_000L
    val target = (player.currentPosition + seekDeltaMs)
        .coerceIn(0L, player.duration.coerceAtLeast(0L))
    player.seekTo(target)
    updateMediaInfo()
    onUserActivity()

    val totalSeconds = (doubleTapSeekAccumulatedMs / 1000L).toInt()
    _uiState.update {
        it.copy(
            seekFeedback = SeekFeedback(isForward, totalSeconds, System.currentTimeMillis())
        )
    }
    seekFeedbackJob?.cancel()
    seekFeedbackJob = viewModelScope.launch {
        delay(900L) // Interaction window
        _uiState.update { it.copy(seekFeedback = null) }
        doubleTapSeekAccumulatedMs = 0L
        doubleTapLastIsForward = null
    }
}
```

8.3 Playback State Machine & Lifecycle Release

ExoPlayer reports its internal state via `Player.Listener`. Falcon Player maps these states to its domain `PlaybackState` interface:

- `STATE_IDLE` -> `PlaybackState.Idle`
- `STATE_BUFFERING` -> `PlaybackState.Buffering` (triggers custom pulsing indicator)
- `STATE_READY` -> `PlaybackState.Playing` if `isPlaying`, else `Paused`
- `STATE_ENDED` -> `PlaybackState.Ended` (saves resume position to 0L, advances playlist if present)

When the user leaves the player, `ViewModel.onCleared()` persists the playback position and invokes `player.release()` to free hardware decoders immediately.

Chapter 9 - Audio Playback and Audio Library

9.1 The AudioPlaybackManager Architecture

Audio playback requires an independent, application-scoped lifecycle so that playback can continue when users navigate between tabs. In `data/AudioPlaybackManager.kt`, an `@Singleton` `ExoPlayer` instance is initialized on the `Application Context`:

```
// Location: app/src/main/java/com/example/falconplayer/data/AudioPlaybackManager.kt
val player: ExoPlayer = ExoPlayer.Builder(context).build().apply {
    val audioAttributes = AudioAttributes.Builder()
        .setContentType(C.AUDIO_CONTENT_TYPE_MUSIC)
        .setUsage(C.USAGE_MEDIA)
        .build()
    setAudioAttributes(audioAttributes, true) // Automatically manages Audio Focus!
    setHandleAudioBecomingNoisy(true) // Pauses automatically if headphones disconnect!
}
```

Audio Focus & Becoming Noisy: Passing `true` to `setAudioAttributes` ensures Falcon Player automatically ducks or pauses when phone calls or navigation directions occur. Setting `setHandleAudioBecomingNoisy(true)` registers an internal `BroadcastReceiver` that pauses playback immediately if Bluetooth or wired headphones are disconnected, avoiding embarrassing loud playback in public environments.

9.2 Queue Management, Shuffle, and Repeat Modes

`AudioPlaybackManager` maintains an `originalQueue` and an `active queue`. When the user enables `shuffle` (`toggleShuffle()`), the current track is retained as index 0, and all remaining tracks are shuffled around it, preventing the currently playing song from restarting. Three repeat modes are supported: `OFF` (stops at queue end), `ALL` (loops back to track 0), and `ONE` (seeks to 0 and replays current track).

Chapter 10 - Audio Tracks and Subtitle System

10.1 Track Discovery & Metadata Extraction

When ExoPlayer finishes parsing media containers (such as MKV or MP4), it triggers `onTracksChanged(tracks: Tracks)`. In `PlayerViewModel.extractTracks()`, Falcon Player scans all track groups:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerViewModel.kt
for (groupIndex in 0 until tracks.groups.size) {
    val group = tracks.groups[groupIndex]
    if (group.type == C.TRACK_TYPE_AUDIO) {
        for (trackIndex in 0 until group.length) {
            val format = group.getTrackFormat(trackIndex)
            val displayName = formatAudioTrackName(format, audioCount)
            val technicalDetails = formatAudioTechnicalInfo(format) // Codec, 5.1/Stereo, kHz
            // Create TrackItem...
        }
    } else if (group.type == C.TRACK_TYPE_TEXT) {
        for (trackIndex in 0 until group.length) {
            val format = group.getTrackFormat(trackIndex)
            val displayName = formatSubtitleTrackName(format, subtitleCount)
            val technicalDetails = formatSubtitleTechnicalInfo(format) // SRT, ASS, Forced, SDH
            // Create TrackItem...
        }
    }
}
```

10.2 Track Switching via TrackSelectionOverride

To switch audio or subtitle tracks without reloading the media file or resetting playback position, Falcon Player builds a `TrackSelectionOverride`:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerViewModel.kt
player.trackSelectionParameters = player.trackSelectionParameters
    .buildUpon()
    .setTrackTypeDisabled(C.TRACK_TYPE_TEXT, false)
    .setOverrideForType(
        TrackSelectionOverride(track.group, listOf(track.trackIndex))
    )
    .build()
```

To disable subtitles completely, Falcon Player calls `setTrackTypeDisabled(C.TRACK_TYPE_TEXT, true)` and `clearOverridesOfType(C.TRACK_TYPE_TEXT)`, instantly clearing on-screen text overlays.

Chapter 11 - Timeline, Controls, Gestures, and Fullscreen

11.1 Custom Interactive Timeline Scrubbing Canvas

Standard Android sliders or Compose Slider components can feel clunky and visually unrefined for video scrubbing. Falcon Player creates a custom, high-precision timeline in `ui/player/components/VideoTimeline.kt` using Compose Canvas and pointer input detection:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/components/VideoTimeline.kt
val trackHeight by animateDpAsState(if (isDragging) 5.5.dp else 3.5.dp)
val thumbRadius by animateDpAsState(if (isDragging) 8.dp else 5.dp)

Canvas(modifier = Modifier.fillMaxWidth().height(trackHeight)) {
    // 1. Inactive full duration track (semi-transparent white)
    drawRoundRect(color = Color.White.copy(alpha = 0.22f), size = Size(size.width, size.height))
    // 2. Buffered progress track (brighter white)
    if (bufferedFraction > 0f) {
        drawRoundRect(color = Color.White.copy(alpha = 0.45f), size = Size(size.width * bufferedFraction,
            size.height))
    }
    // 3. Played progress track (Falcon Red)
    if (playedFraction > 0f) {
        drawRoundRect(color = FalconRed, size = Size(size.width * playedFraction, size.height))
    }
    // 4. Dual-layer Scrubber Thumb (outer Falcon Red ring, inner crisp white core)
    drawCircle(color = FalconRed, radius = thumbRadius.toPx(), center = thumbCenter)
    drawCircle(color = Color.White, radius = thumbRadius.toPx() * 0.4f, center = thumbCenter)
}
```

Touch Ergonomics: The `VideoTimeline` wraps the `Canvas` in a 36dp touch target box with both `detectTapGestures` (for immediate jumps) and `detectDragGestures` (for smooth scrub previewing). The track height expands dynamically from 3.5dp to 5.5dp when the user touches the bar, providing tactile visual feedback.

11.2 Vertical Drag Gestures: Brightness and Volume HUDs

Inside `PlayerScreen.kt`, vertical swipe gestures on the screen surface dynamically adjust device brightness and media volume:

```
// Location: app/src/main/java/com/example/falconplayer/ui/player/PlayerScreen.kt
detectVerticalDragGestures(
onDragStart = { offset ->
if (offset.x < size.width * 0.45f) { // LEFT HALF -> Brightness
showBrightnessHud = true
} else if (offset.x > size.width * 0.55f) { // RIGHT HALF -> Volume
showVolumeHud = true
}
},
onVerticalDrag = { change, dragAmount ->
change.consume()
val delta = -dragAmount / (size.height * 0.65f)
if (change.position.x < size.width * 0.45f) {
brightnessLevel = (brightnessLevel + delta).coerceIn(0.01f, 1.0f)
activity?.window?.attributes = activity.window.attributes.apply { screenBrightness = brightnessLevel }
} else if (change.position.x > size.width * 0.55f) {
volumeLevel = (volumeLevel + delta).coerceIn(0f, 1f)
val maxVol = audioManager.getStreamMaxVolume(AudioManager.STREAM_MUSIC)
audioManager.setStreamVolume(AudioManager.STREAM_MUSIC, (volumeLevel * maxVol).toInt(), 0)
}
}
})
```

11.3 Troubleshooting: The Fullscreen Orientation Reset Fix

A common bug in mobile video players is that requesting landscape orientation via `activity.requestedOrientation = SCREEN_ORIENTATION_SENSOR_LANDSCAPE` locks the entire application in landscape permanently even after returning to the library.

Root Cause & Falcon Player Solution: In `PlayerScreen.kt`, Falcon Player binds a `DisposableEffect(Unit)` directly to the screen's lifecycle. When the screen exits the Compose backstack, `onDispose` runs unconditionally:

```
activity?.requestedOrientation = ActivityInfo.SCREEN_ORIENTATION_UNSPECIFIED.
```

This ensures the device orientation sensor immediately regains control as the user navigates back to `HomeScreen`.

Chapter 12 - Android Permissions, Security, and Storage

12.1 Granular Runtime Permission Matrix

Falcon Player adapts its permission requests depending on the active Android OS API level:

Permission String	API Level	Architectural Purpose & Fallback Behavior
READ_MEDIA_VIDEO	API 33+ (Android 13+)	Allows discovering local video files through MediaStore without requesting photo or document permissions.
READ_MEDIA_AUDIO	API 33+ (Android 13+)	Allows discovering music tracks, album metadata, and artist collections.
READ_EXTERNAL_STORAGE	API 24 - 32 (maxSdk 32)	Legacy media read access on Android 7.0 through Android 12L.
WRITE_EXTERNAL_STORAGE	API 24 - 29 (maxSdk 29)	Legacy file deletion permission on Android 9 (Pie) and below.
FOREGROUND_SERVICE_MEDIA_PLAYBACK	API 34+ (Android 14+)	Declares foreground service permission for uninterrupted media playback.
POST_NOTIFICATIONS	API 33+ (Android 13+)	Permits displaying playback notifications with lock-screen media controls.

12.2 Privacy & Incognito Mode Architecture

In HistoryRepository.kt, Falcon Player checks DisplaySettingsRepository.settings.value.incognitoMode prior to writing video playback entries. When Incognito Mode is toggled on, playback history recording is completely bypassed, ensuring zero trace of media viewing is stored in falcon_history.json.

Chapter 13 - Architecture and Data Flow

13.1 Unidirectional Data Flow (UDF) Mechanics

Falcon Player strictly adheres to the Unidirectional Data Flow pattern. UI state flows downwards from ViewModels to Composables via immutable StateFlow objects, while user events (clicks, drags, seeks) flow upwards as lambdas.

```
+-----+
| VIEW MODEL |
| _uiState = MutableStateFlow(PlayerUiState) |
| uiState: StateFlow<PlayerUiState> |
+-----+
| Emits Immutable State
v
+-----+
| COMPOSE UI SCREEN |
| val uiState by viewModel.uiState |
| .collectAsStateWithLifecycle() |
| Renders PlayerView, Controls, HUDs... |
+-----+
| Dispatches User Events
v (onPlayPause, onSeek, toggleResize...)
+-----+
| VIEW MODEL ACTIONS |
| fun onSeek(pos) -> player.seekTo(pos) |
| Updates _uiState via .update { it.copy() } |
+-----+
```

13.2 Storage Tiering: DataStore vs JSON Files

Falcon Player employs two distinct persistent storage mechanisms based on data access patterns:

1. **Jetpack DataStore Preferences (PlaybackPositionRepository):** Key-value storage mapping video URI strings to millisecond playback timestamps. Highly optimized for rapid, frequent writes as playback progresses.
2. **Thread-Safe JSON Files (PlaylistRepository, DisplaySettingsRepository, FavoritesRepository, HistoryRepository):** Structured lists serialized via Kotlinx Serialization into context.filesDir. Each repository utilizes a Kotlin Mutex (mutex.withLock { ... }) ensuring that parallel coroutines never corrupt file data during read/write cycles.

Chapter 14 - Dependencies and External Libraries

14.1 Gradle Version Catalog Analysis (libs.versions.toml)

All project dependencies are pinned and centrally managed in gradle/libs.versions.toml:

Dependency / Library	Catalog Version	Purpose & Impact on Application
androidx-media3-exoplayer	1.7.0	Core playback engine for video/audio decoding, buffering, seeking.
androidx-media3-ui	1.7.0	Provides PlayerView, AspectRatioFrameLayout, and subtitle renderers.
androidx-media3-session	1.7.0	MediaSession APIs for system controls and background notification sync.
androidx-compose-bom	2026.03.01	Bill of Materials aligning Compose UI, Foundation, and Material 3 versions.
androidx-navigation3-ui	1.0.1	Latest type-safe navigation framework using NavKey and NavDisplay.
hilt-android	2.60.1	Dagger Hilt dependency injection for repositories, managers, and viewmodels.
androidx-datastore-preferences	1.1.1	Modern replacement for SharedPreferences storing playback resume positions.
kotlinx-serialization-json	1.7.3	Compile-time JSON serialization for playlists, favorites, and settings files.

Chapter 15 - Build System and APK Distribution

15.1 Build Configuration & SDK Targets

From `app/build.gradle.kts`, Falcon Player targets the newest Android platform features while retaining backwards compatibility:

- `compileSdk = 36`: Allows compilation against the latest Android platform APIs.
- `targetSdk = 36`: Enforces modern Android runtime behaviors, scoped storage, and notification permissions.
- `minSdk = 24`: Supports Android 7.0 (Nougat) and higher, covering over 96% of active Android devices globally.
- `JavaVersion.VERSION_17`: Standard JVM bytecode target using Gradle toolchains.

15.2 Building and Installing via ADB

To assemble, verify, and install Falcon Player on a physical device or emulator, execute the following commands:

```
# 1. Clean and assemble Debug APK
./gradlew clean assembleDebug

# 2. Output location of generated APK:
# app/build/outputs/apk/debug/app-debug.apk

# 3. Install onto connected Android device via ADB:
adb install -r app/build/outputs/apk/debug/app-debug.apk

# 4. Launch Falcon Player directly from terminal:
adb shell am start -n com.example.falconplayer/.MainActivity
```

Chapter 16 - Debugging and Troubleshooting

16.1 Logcat Diagnostic Tagging Strategy

Falcon Player incorporates structured logging across all data and playback components. When diagnosing runtime issues in Android Studio Logcat, use these specific package tags:

- tag:VideoRepository: Traces MediaStore queries, row counts, and deletion result codes.
- tag:AudioRepository: Monitors music scanning, album art URI resolution, and deduplication.
- tag:AudioPlaybackManager: Logs audio playback state, queue indices, audio focus gains/losses.
- tag:PlaylistRepository: Inspects JSON load/save operations, validation rejections, and mutex locks.
- tag:BrowseRepository: Tracks file counting, secondary SD card detection, and permission checks.

16.2 Common Technical Issues & Systematic Resolution Procedures

Symptom / Failure	Probable Root Cause	Logical Debugging & Fix Procedure
Playback Error Dialog appears immediately on opening a video	Video codec unsupported by device hardware (e.g. 10-bit AV1 on older SoC) or file URI permission revoked.	Inspect onPlayerError(error: PlaybackException) in PlayerViewModel.kt. Check Logcat for MediaCodecRenderer\$DecoderInitializationException. Verify content URI validity.
Delete fails on Android 11+ without prompt	Attempting direct file deletion instead of launching the OS IntentSender.	Ensure HomeViewModel.deleteVideo() emits deleteIntentSender and that HomeScreen's rememberLauncherForActivityResult receives and launches the IntentSenderRequest.
Screen stays locked in landscape after exiting player	Activity orientation override not reset during navigation transition.	Verify that PlayerScreen.kt includes DisposableEffect(Unit) { onDispose { activity.requestedOrientation = SCREEN_ORIENTATION_UNSPECIFIED } }.
Mini player does not pause when headphones disconnected	setHandleAudioBecomingNoisy disabled or audio attributes missing.	Check AudioPlaybackManager.kt line 57: confirm setHandleAudioBecomingNoisy(true) is invoked on the ExoPlayer builder.
Video thumbnails fail to load or cause OutOfMemoryError	Thumbnail bitmap allocation exceeding heap; LruCache size incorrectly calculated.	Check ThumbnailLoader.kt: ensure cacheSize = maxMemory / 8 and that sizeOf correctly calculates bitmap.byteCount / 1024.

Chapter 17 - Source-Code Reference

17.1 Core Class & Method Reference

This section documents the signatures, responsibilities, and call flows of the application's most critical functions:

1. VideoRepository.getVideos(): List<VideoItem>

- **Location:** com.example.falconplayer.data.VideoRepository
- **Parameters:** None. Runs asynchronously on Dispatchers.IO.
- **Returns:** A list of fully populated VideoItem domain models.
- **Called by:** HomeViewModel.loadMedia(), PlaylistViewModel.getPlaylistDetailState().
- **Calls:** ContentResolver.query(MediaStore.Video.Media.EXTERNAL_CONTENT_URI, ...), calculateResolutionBadge().
- **Side Effects:** Reads media database. No disk writes.
- **Failure Consequence:** If modified incorrectly, the home video library will display empty or crash due to null column indices.

2. VideoRepository.deleteVideo(videoUri: Uri): DeleteResult

- **Location:** com.example.falconplayer.data.VideoRepository
- **Parameters:** videoUri: Uri - Content URI of the video to delete.
- **Returns:** DeleteResult.Success, NeedsPermission(IntentSender), or Failure(reason).
- **Called by:** HomeViewModel.deleteVideo(video).
- **Calls:** MediaStore.createDeleteRequest (API 30+) or ContentResolver.delete.
- **Failure Consequence:** Deletions on Android 11+ will fail silently or throw security exceptions.

3. PlayerViewModel.triggerDoubleTapSeek(isForward: Boolean)

- **Location:** com.example.falconplayer.ui.player.PlayerViewModel
- **Parameters:** isForward: Boolean - true for forward (+10s), false for rewind (-10s).
- **Side Effects:** Immediately updates player.seekTo, aggregates doubleTapSeekAccumulatedMs, emits visual SeekFeedback to _uiState, and cancels/restarts the 900ms reset timer.
- **Called by:** PlayerScreen.kt pointer gesture tap detector.

4. AudioPlaybackManager.playQueue(tracks: List<AudioItem>, startIndex: Int)

- **Location:** com.example.falconplayer.data.AudioPlaybackManager
- **Parameters:** tracks: List<AudioItem>, startIndex: Int = 0.
- **Side Effects:** Clears existing player queue, loads target media URI, starts playback, launches 250ms progress polling coroutine.

Chapter 18 - Project-Based Learning Course

18.1 Course Structure: From Novice to Architect

This project-based curriculum teaches you Android development by exploring, experimenting with, and extending Falcon Player through 6 progressive stages:

Stage	Learning Focus & Modules	Practical Exercise & Verification
Stage A Orientation	Understand module layout, Gradle catalog, Manifest permissions, and entry points.	Read-Only: Trace execution from FalconPlayerApp to MainActivity and MainNavigation.
Stage B UI & Compose	Understand Compose layout trees, state hoisting, colors, and design tokens.	Code Change: Modify FalconRed in Color.kt and observe how all accent controls update dynamically.
Stage C Media Discovery	Learn ContentResolver, MediaStore cursors, and scoped storage rules.	Code Change: Add a new filter in HomeViewModel.kt that only shows videos longer than 60 seconds.
Stage D Playback Engine	ExoPlayer initialization, MediaItem creation, playback state listeners.	Code Change: Add a 3.0x speed option to PlayerSettingsSheet.kt.
Stage E Gestures & Tracks	Custom Canvas drawing, drag gesture calculation, audio track switching.	Code Change: Change double-tap seek delta from 10 seconds to 15 seconds in PlayerViewModel.kt.
Stage F Independent Dev	Create new features, manage DataStore preferences, debug with Logcat.	Capstone: Build an 'A-B Loop' feature that repeats a video between two selected timestamps.

Chapter 19 - Feature Status and Technical Debt

19.1 Feature Implementation Inventory

Based on systematic source code inspection across all directories, the following table presents the confirmed implementation status of Falcon Player features:

Feature / Component	Status in Codebase	Technical Evidence & Notes
Hardware Video Playback	Implemented & Confirmed	Media3 ExoPlayer 1.7.0 in PlayerViewModel and PlayerScreen.
Double-Tap Accumulating Seek	Implemented & Confirmed	triggerDoubleTapSeek in PlayerViewModel.kt with 900ms window.
Audio & Subtitle Track Switching	Implemented & Confirmed	TrackSelectionSheet.kt using TrackSelectionOverride.
Single & Bulk Video Delete	Implemented & Confirmed	Scoped storage createDeleteRequest in VideoRepository.kt.
Audio Background Engine & Focus	Implemented & Confirmed	AudioPlaybackManager.kt with setHandleAudioBecomingNoisy.
Custom Video Playlists	Implemented & Confirmed	JSON storage, reordering, creation, and deletion in PlaylistRepository.kt.
File System Explorer (Browse)	Implemented & Confirmed	Java File scanning with SD card detection in BrowseRepository.kt.
Media Notification Service	Partially Implemented	Permission declared in manifest; session bindings present in manager; notification UI can be extended.
External Subtitle (.srt) File Import	Not Implemented	Currently discovers container-embedded subtitle tracks; external file picker is absent.
Equalizer / Audio DSP Effects	Not Implemented	No audio equalizer DSP UI or AudioEffect integration in current codebase.

Chapter 20 - Glossary, FAQ, and Final Reference

20.1 Technical Glossary

- **ExoPlayer:** An application-level media player for Android, now part of AndroidX Media3, that supports DASH, HLS, SmoothStreaming, and extensive local container formats.
- **Media3:** Google's unified media library combining ExoPlayer, media sessions, and UI controls under androidx.media3.
- **Scoped Storage:** Android's privacy framework restricting direct file path access and requiring MediaStore APIs or IntentSender confirmation to delete files.
- **NavKey:** An AndroidX Navigation 3 type-safe serializable destination identifier replacing legacy URL strings.
- **StateFlow:** A state-holder observable flow from Kotlin Coroutines that emits current and updated state to collectors.

20.2 50 Essential Project-Specific Questions & Answers

The following 50 questions and answers are derived directly from the real Falcon Player codebase to serve as an authoritative technical reference:

Q1: What is the root Application class of Falcon Player and what annotation does it require?

The root application class is `FalconPlayerApp` (located at `app/src/main/java/com/example/falconplayer/FalconPlayerApp.kt`). It extends `Application()` and requires the `@HiltAndroidApp` annotation to trigger Dagger Hilt's code generation for dependency injection.

Q2: Which activity serves as the application entry point?

`MainActivity` (`com.example.falconplayer.MainActivity`), which extends `ComponentActivity` and is annotated with `@AndroidEntryPoint`.

Q3: How does Falcon Player achieve an immersive fullscreen mode on startup?

Inside `MainActivity.onCreate()`, it calls `WindowCompat.getInsetsController(window, window.decorView)`, configures `systemBarsBehavior = BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE`, and calls `hide(systemBars())`.

Q4: Which navigation library does Falcon Player use?

AndroidX Navigation 3 (specifically `androidx.navigation3:navigation3-runtime:1.0.1` and `navigation3-ui`), utilizing `NavDisplay` and `rememberNavBackStack`.

Q5: What are the three NavKey navigation routes defined in the project?

`Home` (data object `Home : NavKey`), `PlaylistDetail(val playlistId: String)`, and `Player(val videoUri: String?, val title: String?, val playlistId: String?, val playlistIndex: Int)`.

Q6: Where is the Media3 ExoPlayer instance created for video playback?

Inside `PlayerViewModel` (`com.example.falconplayer.ui.player.PlayerViewModel`), initialized as `'val player: ExoPlayer = ExoPlayer.Builder(context).build()'`.

Q7: How are custom Compose controls overlaid on top of the video surface?

In `PlayerScreen.kt`, a `Box` container places `AndroidView(PlayerView)` at the lowest z-index, followed by gesture interceptors, HUD layers, and `PlayerControls`.

Q8: Why is `PlayerView.useController` set to false?

To disable Media3's default XML-based playback controls, allowing Falcon Player's custom Compose controls (`PlayerControls.kt`) to manage the UI.

Q9: How does double-tap seeking accumulate seek time across multiple taps?

In `PlayerViewModel.triggerDoubleTapSeek()`, `doubleTapSeekAccumulatedMs` is incremented by 10,000L per tap in the same direction, emitting total seconds to `seekFeedback`. A 900ms delay resets the accumulator.

Q10: How does Falcon Player prevent permanent landscape lock when exiting the player?

`PlayerScreen.kt` uses `DisposableEffect(Unit)` whose `onDispose` block resets `activity.requestedOrientation = ActivityInfo.SCREEN_ORIENTATION_UNSPECIFIED`.

Q11: How does Falcon Player adjust screen brightness during playback?

Vertical drag gestures on the left 45% of `PlayerScreen` modify `activity.window.attributes.screenBrightness` between 0.01f and 1.0f.

Q12: How does Falcon Player adjust audio volume during video playback?

Vertical drag gestures on the right 55% of `PlayerScreen` adjust `AudioManager.STREAM_MUSIC` via `audioManager.setStreamVolume()`.

Q13: Which aspect ratio modes does Falcon Player support?

`VideoResizeMode.FIT` (`RESIZE_MODE_FIT`), `ZOOM` (`RESIZE_MODE_ZOOM`), and `STRETCH` (`RESIZE_MODE_FILL`) via `AspectRatioFrameLayout`.

Q14: How are video tracks (audio and subtitles) discovered by the player?

`PlayerViewModel` registers an `onTracksChanged(tracks: Tracks)` listener on `ExoPlayer`, iterating through track groups and checking `C.TRACK_TYPE_AUDIO` and `C.TRACK_TYPE_TEXT`.

Q15: How are embedded subtitle formats identified in Falcon Player?

In `PlayerViewModel.formatSubtitleTechnicalInfo()`, `format.sampleMimeType` is inspected for WebVTT, SRT (subrip), ASS/SSA, PGS, DVB, and TX3G.

Q16: How does Falcon Player disable subtitles completely?

PlayerViewModel.disableSubtitles() calls
player.trackSelectionParameters.buildUpon().setTrackTypeDisabled(C.TRACK_TYPE_TEXT,
true).clearOverridesOfType(C.TRACK_TYPE_TEXT).build().

Q17: How is playback position saved across app sessions?

PlaybackPositionRepository uses Jetpack DataStore Preferences (name = 'playback_positions'), storing longPreferencesKey(videoUri) to positionMs.

Q18: Under what condition is playback position NOT saved?

If the video duration is zero or the current position is within 5 seconds of the video end, position saving is skipped to prevent resuming at the credits.

Q19: How are video thumbnails extracted and cached?

ThumbnailLoader.kt uses a global LruCache sized to maxMemory / 8. On Android 10+ it uses ContentResolver.loadThumbnail(Size(320, 200)), with MediaMetadataRetriever as fallback.

Q20: What columns are queried from MediaStore for video files?

_ID, DISPLAY_NAME, TITLE, DURATION, WIDTH, HEIGHT, SIZE, BUCKET_ID, BUCKET_DISPLAY_NAME, DATA, and DATE_ADDED in VideoRepository.kt.

Q21: How does Falcon Player group videos into folders?

VideoRepository.getFolders() groups scanned VideoItem records by bucketId (videos.groupBy { it.bucketId }) and maps them to FolderItem.

Q22: How does Falcon Player handle video deletion on Android 11+ (API 30+)?

VideoRepository.deleteVideo() calls MediaStore.createDeleteRequest(context.contentResolver, listOf(videoUri)).intentSender and returns DeleteResult.NeedsPermission.

Q23: How does Falcon Player handle video deletion on Android 10 (API 29)?

It attempts contentResolver.delete(), catches android.app.RecoverableSecurityException, and extracts e.userAction.actionIntent.intentSender.

Q24: How does Falcon Player perform atomic bulk video deletion?

HomeViewModel.deleteSelectedVideos() passes selected URIs to VideoRepository.deleteVideos(), which calls MediaStore.createDeleteRequest with the entire list on Android 11+.

Q25: How does the user activate selection mode in HomeScreen?

By long-pressing any video card via `combinedClickable(onLongClick = { viewModel.enterSelectionModeWithVideo(video.id) })`.

Q26: Where are user-created playlists stored?

Inside `context.filesDir / 'falcon_playlists.json'`, serialized as JSON using Kotlinx Serialization with a Mutex for thread safety.

Q27: What validation rules are applied when creating or renaming a playlist?

`PlaylistRepository.validatePlaylistName()` requires non-empty names, limits length to 50 characters, and enforces case-insensitive uniqueness.

Q28: How does drag-and-drop reordering work in PlaylistRepository?

`PlaylistRepository.moveVideoInPlaylist()` removes the item at `fromIndex`, inserts it at `toIndex`, and persists the reordered list to JSON.

Q29: How does Falcon Player discover audio files in AudioRepository?

It queries `MediaStore.Audio.Media.EXTERNAL_CONTENT_URI` with a filter including `IS_MUSIC != 0`, `MIME_TYPE 'audio/%'`, common extensions (`.mp3`, `.flac`, `.m4a...`), and `duration >= 1000ms`.

Q30: How are audio tracks grouped in AudioScreen?

Into 4 tabs: Artists (grouped by artist), Albums (grouped by `albumId/album`), Tracks (sorted by title), and Genres (grouped by genre name).

Q31: How is audio playback managed separately from video?

Via `AudioPlaybackManager`, a singleton class holding an `ExoPlayer` instance that remains active across navigation tab switches.

Q32: How does Falcon Player automatically handle headphone disconnection?

`AudioPlaybackManager` sets `setHandleAudioBecomingNoisy(true)` on its `ExoPlayer` instance, automatically pausing audio when headphones unplug.

Q33: How does Falcon Player handle Audio Focus for music playback?

`AudioPlaybackManager` sets `AudioAttributes` with `USAGE_MEDIA` and `CONTENT_TYPE_MUSIC` and passes `handleAudioFocus = true` to `setAudioAttributes`.

Q34: How does shuffle work in AudioPlaybackManager without interrupting the current track?

`AudioPlaybackManager.toggleShuffle()` keeps the current track at index 0 and shuffles only the remaining tracks from `originalQueue`.

Q35: What repeat modes are supported in AudioPlaybackManager?

AudioRepeatMode.OFF (stops at end), AudioRepeatMode.ALL (loops back to track 0), and AudioRepeatMode.ONE (seeks to 0 and replays current track).

Q36: What is the AudioMiniPlayer?

A floating Composable bar anchored above the bottom NavigationBar in HomeScreen that displays current song title, artist, album art, play/pause, next, and a progress bar.

Q37: What is the BrowseScreen?

A file manager screen in Falcon Player that allows users to explore device storage directories directly, with quick access to Download, Movies, and Music folders.

Q38: How does BrowseRepository detect secondary storage (SD cards)?

By scanning subdirectories in '/storage' for readable directories other than 'emulated' and 'self'.

Q39: What is Incognito Mode in Falcon Player?

A toggleable mode stored in falcon_display_settings.json. When active, HistoryRepository.recordVideoPlayed() skips recording video playback entirely.

Q40: What sorting options are supported in DisplaySettingsRepository?

SortType: NAME_ASC, NAME_DESC, LENGTH_ASC, LENGTH_DESC, ADDED_ASC, ADDED_DESC, TRACKS_DESC, TRACKS_ASC, INSERTION_ASC, INSERTION_DESC.

Q41: How does Falcon Player toggle between Grid and List video layouts?

DisplaySettingsRepository stores isListView. HomeScreen dynamically switches between RealVideoGridCard and RealVideoListCard.

Q42: What is the primary brand accent color of Falcon Player?

FalconRed (Color(0xFFE50914)), inspired by high-contrast crimson aesthetics.

Q43: How is the Falcon brand logo drawn?

Via FalconLogo.kt using Compose Canvas, drawing 3 wing feathers, a gradient aerodynamic beak, crest highlight, and a piercing white eye.

Q44: What minimum Android SDK version does Falcon Player require?

minSdk = 24 (Android 7.0 Nougat).

Q45: What target and compile SDK versions are configured?

compileSdk = 36 and targetSdk = 36.

Q46: Which Java toolchain version is used?

Java 17 (jvmToolchain(17)).

Q47: What dependency injection framework is used?

Dagger Hilt (com.google.dagger:hilt-android:2.60.1) processed via KSP (Kotlin Symbol Processing).

Q48: How does Picture-in-Picture (PiP) work in PlayerScreen?

When user taps the PiP button, `context.enterPictureInPictureMode(PictureInPictureParams.Builder().build())` is called on Android 8.0+ (API 26+).

Q49: How are unit tests structured in the project?

In `app/src/test/java/com/example/falconplayer/ui/player/PlayerUiStateTest.kt`, testing default media info initialization and `PlaybackState.statusText` mappings.

Q50: How do you build the debug APK using Gradle wrapper?

Run `./gradlew assembleDebug`. The APK is generated at `'app/build/outputs/apk/debug/app-debug.apk'`.

20.3 Long-Term Developer Mastery Roadmap

To advance your mastery of Android systems engineering using Falcon Player, proceed through these 4 architectural milestones:

- **Phase 1: MediaCodec & ExoPlayer Internals:** Study ExoPlayer's MediaCodecRenderer and DefaultRenderersFactory to understand how hardware buffers flow from decoders to SurfaceViews.
- **Phase 2: Custom DataSource & Caching:** Extend Media3 with CacheDataSource and SimpleCache to support streaming video playback with offline chunk caching.
- **Phase 3: Foreground Media Session:** Upgrade AudioPlaybackManager to a full AndroidX MediaSessionService providing lock-screen media notifications and Android Auto support.
- **Phase 4: Room Database Migration:** Transition PlaylistRepository from JSON files to an encrypted SQLite Room database with foreign key cascades and reactive Paging 3 queries.